ICPC Latin American Regional Contests
Gran Premio de México 2026

JURASSIC PARSE

Documentación Oficial — Guía Introductoria ICPC

Competencia: Gran Premio de México — Clasificatorio
Lenguaje: C++20 (g++ 13.3.0) | Ubuntu 24.04

El proposito de este documento es que cualquier miembro del equipo pueda leerlo y sepa exactamente qué hacer, cómo funciona el sistema y cuál es la estrategia.

1. ¿Qué es el ICPC?

El ICPC (International Collegiate Programming Contest) es la competencia de programación universitaria más importante del mundo. Equipos de tres personas resuelven problemas algorítmicos en un tiempo limitado usando una sola computadora.

1.1 Estructura del concurso

El concurso se divide en niveles. Nosotros comenzamos en el Gran Premio de México (clasificatorio regional). El camino completo es:

Nivel	Descripción
Gran Premio de México	Clasificatorio. Varias fechas, compites en línea o en sede.
Final Mexicana	Los mejores del Gran Premio compiten por representar a México.
LAR (Regional Latinoamericana)	Los mejores de México compiten con toda Latinoamérica.
LAC (Championship)	Los mejores de LAR. Para 2025-26, N=40 equipos clasifican.
World Finals	Los mejores de todo el mundo. Cima del ICPC.

1.2 Formato del concurso

- Duración: 5 horas.
- Equipo: 3 integrantes, 1 sola computadora.
- Problemas: típicamente entre 10 y 13 problemas por concurso, de dificultad variada.
- Gana el equipo que resuelve más problemas. En caso de empate, gana el que acumuló menos tiempo de penalización.
- No hay comunicación con otros equipos durante el concurso.

2. Puntaje y Penalizaciones

El puntaje se basa en dos criterios principales: cuántos problemas resuelves y cuánto tiempo tardaste.

2.1 Cómo se calcula el tiempo

El tiempo se cuenta en minutos desde el inicio del concurso hasta el momento del primer AC (accepted) en cada problema. La suma de esos tiempos es tu tiempo total. Por ejemplo, si resuelves el problema A a los 40 mín y el B a los 90 mín, tu tiempo base es 130 mín.

2.2 Penalizaciones por intentos fallidos

Cada *submission* con veredicto NO (excepto Compilation Error y Contact Staff) añade 20 minutos de penalización al tiempo final de ese problema. Los problemas no resueltos no generan penalización.

REGLA CLAVE	Solo se penaliza si al final sí resolviste el problema. Si nunca lo resuelves, los intentos no cuentan.
Compilation Error	No genera penalización. Corrige y vuelve a intentar sin costo.
Contact Staff (6)	No genera penalización. Se usa para casos imprevistos.
Wrong Answer	+20 min por intento, si el problema es eventualmente resuelto.
Time Limit Exceeded	+20 min por intento, si el problema es eventualmente resuelto.

2.3 Verdicts (respuestas del juez)

Veredicto	Significado
1 - YES	¡Correcto! La solución pasó todos los casos de prueba.
2 - NO - Compilation error	El código no compiló. Sin penalización.
3 - NO - Runtime error	El programa falló en ejecución (segfault, división entre cero, etc.).
4 - NO - Time limit exceeded	La solución tardó demasiado. Optimiza el algoritmo.
5 - NO - Wrong answer	La respuesta es incorrecta en algún caso de prueba.
6 - NO - Contact staff	Caso imprevisto. Ve a Clarifications con el número de run.

3. Entorno Técnico

Todos los equipos en el LAR 2025 compiten bajo el mismo entorno.

3.1 Sistema operativo y compiladores

Sistema Operativo	Ubuntu GNU/Linux 24.04 LTS amd64
C / C++20	gcc/g++ version 13.3.0
Python 3	PyPy3 version 7.3.15 (Python 3.9.18)
Java	OpenJDK 21.0.8
Kotlin	kotlin version 2.1.10 (JRE 21.0.8)

3.2 Límites del sistema

Límite	Valor
RAM total	1 GB (incluye el runtime del lenguaje)
Stack máximo	128 MB
Tamaño del archivo fuente	100 KB
Tamaño del ejecutable compilado	32 MB
Tiempo de compilación	10 segundos (Puede variar según el problema)

Nota importante: El límite de 1 GB incluye la memoria usada por el entorno de ejecución. En Java, Python y Kotlin, el runtime ya consume varios MB. En C/C++ el overhead es mínimo.

3.3 Comandos de compilación y ejecución

Usa siempre estos comandos — son idénticos a los que usa el juez:

Lenguaje	Comando
Compilar C	gcc -x c -g -O2 -std=gnu11 -static {archivo} -lm
Compilar C++	g++ -x c++ -g -O2 -std=gnu++20 -static {archivo}
Compilar Java	javac -sourcepath . -d . {archivo}
Ejecutar Java	java -XX:+UseSerialGC -Xms1024m -Xmx1024m -Xss128m {clase}
Verificar Python	pypy3 -m py_compile {archivo}
Ejecutar Python	pypy3 {archivo}
Compilar Kotlin	kotlinc -d . {archivo}
Ejecutar Kotlin	kotlin -J-XX:+UseSerialGC -J-Xms1024m -J-Xmx1024m -J-Xss128m {clase}Kt

3.4 Reglas importantes por lenguaje

C++

- Agrega siempre ios::sync_with_stdio(0); cin.tie(0); al inicio del main para inputs grandes.
- No mezcles scanf/printf con cin/cout cuando deshabilitas la sincronización.
- El programa debe terminar con return 0 o exit(0).

Java

- NO declares package. Si declaras package, el programa no ejecuta en BOCA.
- El nombre de la clase pública debe ser una letra mayúscula (A, B, C...) según el problema.
- Usa BufferedReader y PrintWriter en vez de Scanner para inputs grandes.

Python

- Se ejecuta con PyPy3, no CPython. El comportamiento es muy similar pero puede haber diferencias sutiles.
- Si el código falla la verificación de sintaxis, recibes Compilation Error.

Kotlin

- NO declares package. Mismo motivo que Java.
- El archivo fuente debe nombrarse con letra mayúscula (A.kt, B.kt...).

4. Sistema de Submission (BOCA)

BOCA es el sistema web que usan todos los concursos LAR. Todo se hace desde ahí: enviar soluciones, preguntar al juez, ver el scoreboard e imprimir archivos.

4.1 Cómo enviar una solución

- Abre el navegador y accede a BOCA con el usuario y contraseña del equipo.
- Ve a la pestaña Runs.
- Selecciona el problema correcto y el lenguaje usado.
- Sube el archivo fuente y espera el veredicto.

Tu programa puede ejecutarse en múltiples archivos de entrada. Si tiene más de un error (por ejemplo TLE y WA), puedes recibir cualquiera de los dos como veredicto.

4.2 Reglas de formato de salida

- No existe 'Presentation Error'. Si escribes mal una palabra requerida como 'impossible', es Wrong Answer.
- Mayúsculas/minúsculas generalmente no importan salvo que el problema lo especifique. 'Yes' y 'yes' son equivalentes, pero 'yse' es Wrong Answer.
- Espacios extra al final de una línea o entre tokens son aceptables. Miles de espacios en blanco no.
- Para salida de punto flotante, se acepta cualquier respuesta dentro de la tolerancia especificada en el problema (absoluta y/o relativa). No necesitas que los dígitos coincidan exactamente.
- Si el problema acepta múltiples respuestas válidas, el juez verifica que tu salida cumpla el formato y las restricciones, no que sea idéntica al sample.
- Output excesivo genera Runtime Error. Los mensajes en stderr también cuentan hacia ese límite — reduce tus prints de debug.

4.3 Clarificaciones

Para hacer preguntas al juez durante el concurso:

- Ve a la pestaña Clarifications en BOCA.
- Selecciona el problema y escribe tu pregunta.
- Las respuestas del juez aparecen ahí. Habrá una alerta cuando te respondan.

Importante: la mayoría de las preguntas ya están respondidas en el enunciado del problema. Lee cuidadosamente antes de preguntar.

4.4 Scoreboard y tareas

- Scoreboard: pestaña Score en BOCA. Muestra el ranking local del concurso.
- Imprimir archivos: pestaña Tasks → selecciona el archivo → Send.
- Pedir ayuda técnica (computadora, hardware): pestaña Tasks → botón S.O.S. Solo para problemas físicos, no de programación.

5. Problemas Interactivos

Algunos problemas en el set pueden ser interactivos. En estos, el programa se comunica con otro programa (el interactor) en tiempo real: envías datos y recibes respuestas.

5.1 La diferencia clave: flush

En un problema normal escribes toda la salida y terminas. En un problema interactivo necesitas enviar (flush) cada mensaje inmediatamente después de escribirlo. Si no haces flush, la comunicación se bloquea y recibirás TLE aunque tu algoritmo sea correcto.

Lenguaje	Cómo hacer flush
C (con cstdio)	fflush(stdout);
C++ con endl	endl hace flush automáticamente. Para otros casos: cout.flush();
Java / Kotlin	System.out.flush(); o usar PrintWriter con autoFlush activado.
Python	sys.stdout.flush() o print(..., flush=True)

5.2 Reglas de los problemas interactivos

- El enunciado define el protocolo exacto: quién empieza, formato de cada mensaje, cómo termina la interacción. Síguelo al pie de la letra.
- Si tu programa intenta leer cuando no hay nada disponible (porque olvidaste hacer flush), se quedará esperando indefinidamente — TLE.
- Outputs malformados o en orden incorrecto pueden hacer que el interactor termine con Wrong Answer.
- El interactor puede ser adversarial: adapta sus respuestas para exponer errores en tu algoritmo. El enunciado especificará si es adversarial o determinístico.
- El tiempo límite ya incluye el trabajo del interactor. Si tienes que resolver en a lo más X consultas, esas interacciones deben realizarse dentro del mismo límite de tiempo.

6. Estrategia de Concurso

Un equipo bien coordinado resuelve más problemas que uno con mejor conocimiento técnico pero mala comunicación. Este protocolo debe seguirse en todos los simulacros.

6.1 Timeline de las 5 horas

Tiempo	Fase	Descripción
0:00 – 0:15	**Primera lectura**	Los 3 leen TODOS los problemas en paralelo. Anotan dificultad estimada: Fácil / Medio / Difícil.
0:15 – 0:30	**Clasificación**	Normalmente hay 2 o 3 problemas más fáciles llamados *guaranteed solves* es necesario identificarlos lo mas rápido posible, luego 1-2 de dificultad media. Ignorar los imposibles por ahora.
0:30 – 2:00	**Ataque inicial**	El programador más rápido ataca el problema más fácil en la PC. Los otros 2 resuelven en papel. Objetivo: 2-3 AC antes de la hora 2.
2:00 – 3:30	**Momentum**	Rotar en la PC según quién tenga solución lista. Si alguien está bloqueado 20+ min: cambiar de problema. Mantener el scoreboard en mente.
3:30 – 4:30	**Problemas difíciles**	Atacar el problema más difícil que parezca abordable. Uno piensa en voz alta, los otros validan.
4:30 – 5:00	**Cierre**	Intentar submissions pendientes. Revisar problemas con WA. Últimos 15 min: NO empezar nada nuevo. Solo limpiar lo que está casi listo.

6.2 Reglas de oro

- NUNCA pasar más de 40 min seguidos en un problema sin resultados. En ese caso se debe de cambiar de problema o pedir ayuda al resto del equipo.
- SIEMPRE validar el *approach* con el equipo antes de empezar a codear algo complejo (a excepción de estar muy seguro de lo que se hace, cosa que es rara).
- FAST AC: cuando alguien tiene una solución clara y corta, a la PC inmediatamente.
- PAPEL: los dos que no están en la PC siempre resuelven en papel. No desperdicien tiempo.
- CALMA: un WA no es el fin. Leer el problema de nuevo lentamente suele revelar el edge case.
- SCOREBOARD: revisarlo en el minuto 30, 90 y 150. Ajustar estrategia según la posición.

6.3 Clasificación rápida de problemas

Al leer un problema por primera vez, clasifícalo en menos de 2 minutos usando estas señales:

Señal en el enunciado	Algoritmo probable
n ≤ 20, subconjuntos	Bitmask DP / Backtracking
n ≤ 500, O(n²) ok	DP clásica O(n²) / Floyd-Warshall
n ≤ 10^5 con queries	Segment Tree / BIT / DSU
Grafo con pesos	Dijkstra / Bellman-Ford / MST
'Mínimo de máximo'	Búsqueda binaria + verificador
Cadenas / patrones	KMP / Z-function / Hashing
Maximizar asignaciones	Matching / Flujo máximo
Geometría (área, intersección)	Sweep line / Convex hull

7. Plan de Estudio — 4 Semanas

Programa de preparación del 19 de marzo al 18 de abril de 2026. 2-3 horas diarias estructuradas. Fortaleza del equipo: lógica pura. Áreas de enfoque: Programación Dinámica y matemáticas avanzadas.

7.1 Prioridad de temas

Prioridad	Impacto	Temas
ESENCIAL	~70% de problemas	Sorting, Búsqueda binaria, BFS/DFS, DP clásica, Matemáticas básicas
ALTO	5 → 8 problemas	Grafos avanzados, Segment Tree, Union-Find, Flujo máximo
COMPETITIVO	Top 20%	Geometría, Strings avanzados, Teoría de números

7.2 Programa semanal

Fase 1 — Diagnóstico y Fundamentos (19–25 Mar)

Día	Actividad
Día 1–2	Diagnóstico DP: resolver 8-10 clásicos (knapsack, LIS, LCS) con cronómetro para identificar brechas.
Día 3–4	DP reparación: reforzar subtipos que fallaron + 5 problemas CF Div2-B/C.
Día 5	Grafos base: BFS/DFS — componentes, distancias, grids. 3-4 problemas CF.
Día 6	Matemáticas esenciales: aritmética modular, potencia rápida, criba de Eratóstenes.
Día 7	Repaso mixto: varios temas días 1-6, refuerzo de lo más débil. Mínimo 4 problemas CF.

Fase 2 — Consolidación (26 Mar – 1 Abr)

Día	Actividad
Día 8–9	Grafos avanzados: Dijkstra, Bellman-Ford, Floyd-Warshall. 4 problemas CF Div2-C/D.
Día 10–11	MST + Union-Find: Kruskal, Prim, DSU con path compression. 3-4 problemas.
Día 12	DP intermedia: Bitmask DP, DP en árboles. 3 problemas CF.
Día 13	Segment Tree básico: point update + range query, intro a lazy propagation.
Día 14	Repaso mixto: simulacro individual 2h + sesión de equipo de comunicación de soluciones.

Fase 3 — Entrenamiento avanzado (2–8 Abr)

Día	Actividad
Día 15–16	Strings: KMP, Z-function, hashing. 3-4 problemas CF Div2-D.
Día 17–18	DP avanzada: CHT intro, DP con estructuras. 3 problemas.
Día 19–20	Problemas híbridos: DP+Greedy, DP+Grafos — zona de ventaja con lógica pura.

Día 21	Primer simulacro ICPC cronometrado en equipo (2h, 5 problemas).

Fase 4 — Afinación pre-contest (9–18 Abr)

Día	Actividad
Día 22–24	3 sesiones completas de 5h en equipo. Analizar errores y brechas post-simulacro.
Día 25–26	Coordinación de equipo: afinar quién ataca qué, protocolo de comunicación, gestión del tiempo.
Día 27–28	Repaso dirigido: solo los 2 temas con más errores. Revisar plantillas.
Día 29–30	Descanso activo: sin problemas nuevos. Repaso mental y descanso real. Llegar fresco al 18.

7.3 Rutina diaria recomendada

Tiempo	Actividad
30 min	Teoría / Lectura del tema del día (CPH o guía ICPC).
60–90 min	Resolución de 3–5 problemas en Codeforces / AtCoder.
30 min	Revisión de editorial si no se resolvió en 30 min por problema.
Sábados	Repaso mixto de semana + sesión de equipo (coordinación).

8. Referencia Rápida — Día del Concurso

Esta sección es para revisar los 5 minutos antes de entrar. Todo lo que necesitas recordar.

8.1 Checklist de inicio

- Conecta tu cuenta en BOCA y verifica que puedes ver las pestañas Runs, Clarifications y Score.
- Verifica que el compilador funciona con un hello world antes del concurso.
- Define quién empieza a leer qué problema (los 3 en paralelo desde el minuto 0).
- Ten listo el template base con fast I/O antes de que inicie.

8.2 Ambiente técnico resumido

Compilador	g++ 13.3.0 / C++20 / Ubuntu 24.04 LTS
Comando C++	g++ -x c++ -g -O2 -std=gnu++20 -static {archivo}
Fast I/O	ios::sync_with_stdio(0); cin.tie(0); — SIEMPRE en main
Límite RAM	1 GB total (incluye runtime)
Límite Stack	128 MB
Fuente máxima	100 KB
Penalización	+20 min por NO (excepto CE y Contact Staff)

8.3 Si algo falla

Situación	Qué hacer
Compilation Error inesperado	Verifica que no declaras package (Java/Kotlin). Revisa el comando exacto de compilación.
Runtime Error extraño	Revisa accesos fuera de límites, stack overflow, división entre cero. Reduce prints en stderr.
TLE en solución correcta	Agrega fast I/O. Verifica complejidad. Considera si PyPy3 es mejor opción.
WA en todos los casos	Vuelve a leer el problema desde el inicio. Revisa edge cases: n=0, n=1, valores máximos.
Problema interactivo	Verifica que haces flush después de cada print. Sin flush = TLE garantizado.
Necesitas ayuda del staff	BOCA → Clarifications con número de run. Para hardware: Tasks → S.O.S.

Jurassic Parse — Three Guys One Keyboard — ICPC México 2026

Documentación Oficial del Equipo